
Amazon DynamoDB Indexes: LSI vs GSI

What are Indexes in DynamoDB?

Indexes in DynamoDB let you **query data using attributes other than the primary key** (Partition Key + Sort Key).

 Think of an index as a "different way to look up data" — just like an index in a book helps you find topics quickly.

There are **two types** of secondary indexes in DynamoDB:

-  **LSI** – Local Secondary Index
-  **GSI** – Global Secondary Index

1. Local Secondary Index (LSI)

-  **LSI allows an alternate sort key** on the same **partition key**.

Key Properties:

Feature	Description
 Partition Key	Same as the base table
 Sort Key	Different (alternate)
 Tied to Table	Yes – must be defined at table creation



Consistency

Supports **strongly consistent** and eventually consistent reads



Max per table

5 LSIs per table



Example:

Imagine a **Student table** with:

- Partition Key: `StudentID`
- Sort Key: `Subject`

You define an LSI on:

- Partition Key: `StudentID`
- Alternate Sort Key: `ExamDate`

Now you can:

- Query all exams a student took **sorted by date**.

```
{  
  "StudentID": "1234",  
  "Subject": "Math",  
  "ExamDate": "2024-06-01",  
  "Score": 88  
}
```



2. Global Secondary Index (GSI)

♦ **GSI allows querying on completely different keys** — even unrelated to the base table's primary key.



Key Properties:

Feature	Description
 Partition Key	Can be different from base table
 Sort Key	Optional
 Tied to Table	No – can be added anytime
 Consistency	Eventually consistent only (no strong consistency)
 Max per table	20 GSIs per table

Example:

For the same **Student** table, you want to search by **ExamDate**.

Define GSI:

- Partition Key: **ExamDate**
- Sort Key: **StudentID** (optional)

Now you can:

- Query all students who took exams on **2024-06-01**

LSI vs GSI – Side-by-Side Comparison

Feature 	LSI 	GSI 
Partition Key	Same as base table	Can be different
Sort Key	Alternate from base table	Optional
Created at	Table creation time only	Can be added later
Consistency	Strong or eventual	Eventual only

Max per table	5	20
Query flexibility	Limited to same partition	Much more flexible
Use Case	Sort same partition differently	Query across all partitions

Use Cases

Use Case	Index Type
Query orders for a customer by order date	LSI
Find all users by email address (not in PK)	GSI
Sort blog posts by category and timestamp	GSI
Lookup all products by SKU and price	GSI or LSI depending on key design

Querying with Indexes (CLI Example)

Query with GSI:

```
aws dynamodb query \
  --table-name Student \
  --index-name ExamDateIndex \
  --key-condition-expression "ExamDate = :date" \
  --expression-attribute-values '{":date":{"S":"2024-06-01"}}'
```

Best Practices for Using Indexes

- ✓ Use **LSIs** when your access pattern involves **sorting within the same partition**
 - ✓ Use **GSIs** when you need **completely different access patterns**
 - ✓ Always monitor **GSI WCU/RCU usage** — they are billed separately
 - ✓ **Project only necessary attributes** to minimize storage and cost
 - ✓ Avoid **hot partitions** in GSIs by choosing a well-distributed partition key
-

Summary Table

Index Type	Partition Key	Sort Key	Add Anytime?	Consistency	Use When...
 LSI	Same as table	Yes	 No	 Strong/Eventually	Sorting by different field
 GSI	Different	Optional	 Yes	 Eventually Only	Alternate query patterns
